

Algorithms

Lecture # 33

Syed Hamed Raza

Strong Components

Strong Components

Strong Components

- We consider an important connectivity problem with digraphs.
- When digraphs are used in communication and transportation networks, people want to know that their networks are *complete*.
- Complete in the sense that that it is possible from any location in the network to reach any other location in the digraph.

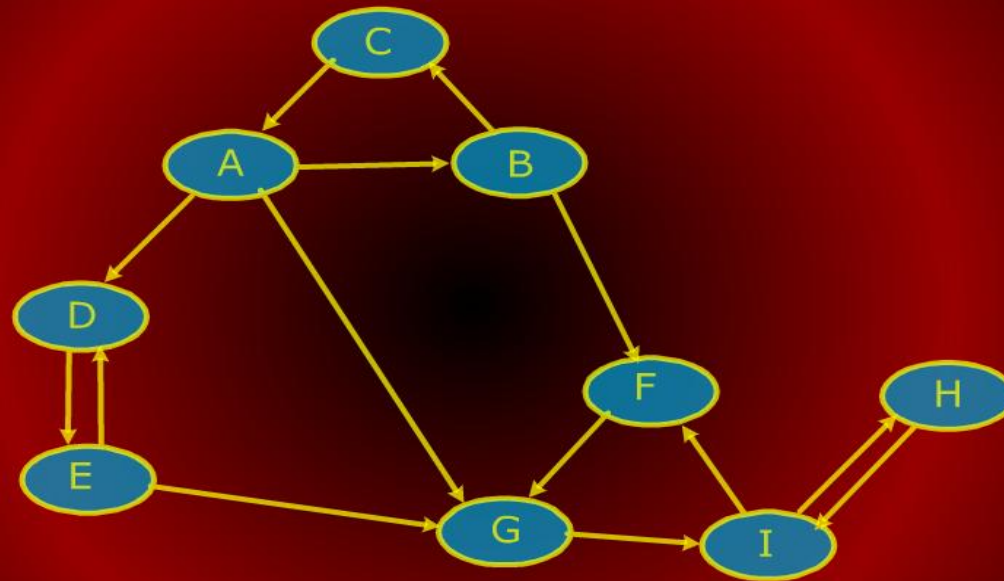
Strong Components

Strong Components

- A digraph is *strongly connected* if for every pair of vertices $u, v \in V$, u can reach v and vice versa.
- We would like to write an algorithm that determines whether a digraph is strongly connected.
- In fact, we will solve a generalization of this problem, of computing the *strongly connected components* of a digraph.

Strong Components

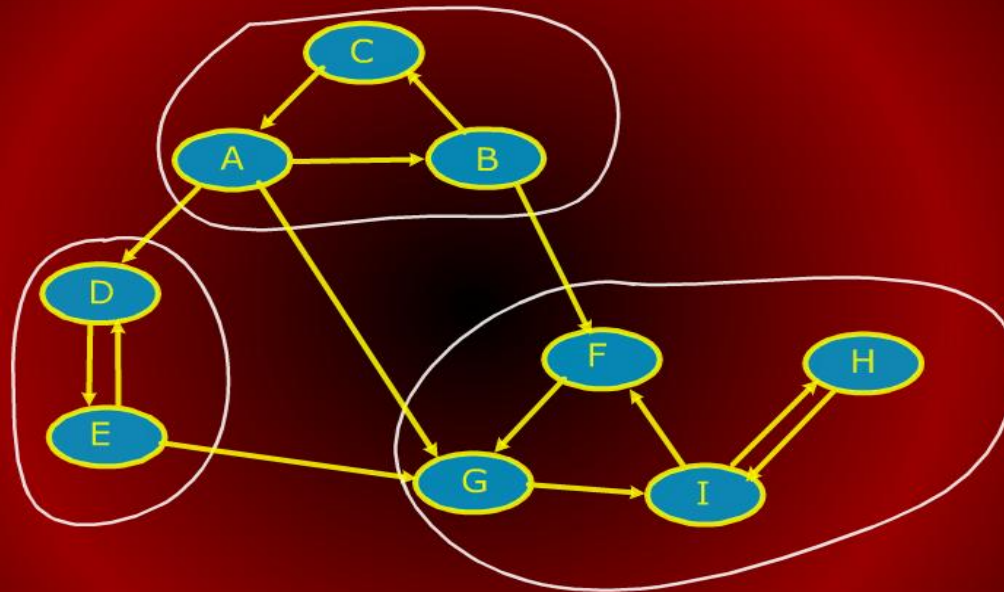
Strong Components



Digraph

Strong Components

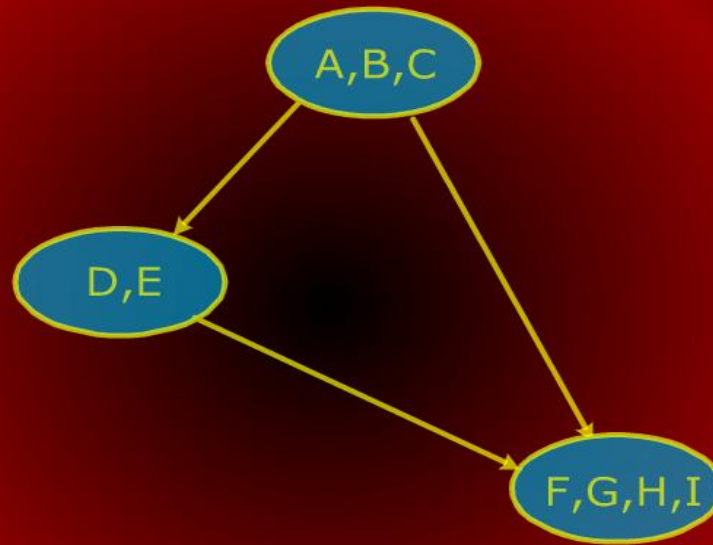
Strong Components



Digraph and Strong Components

Strong Components

Strong Components



Component DAG

Strong Components

Strong Components

- We partition the vertices of the digraph into subsets such that the induced subgraph of each subset is strongly connected.
- We say that two vertices u and v are *mutually reachable* if u can reach v and vice versa.

Strong Components

Strong Components

- It is easy to see that mutual reachability is an *equivalence relation*.
- This equivalence relation partitions the vertices into equivalence classes of mutually reachable vertices and these are the strong components.

Strong Components

Strong Components

- If we merge the vertices in each strong component into a single *super vertex*,
- and join two super vertices (A, B) if and only if there are vertices $u \in A$ and $v \in B$ such that $(u, v) \in E$,
- then the resulting digraph is called the *component digraph*.
- The component digraph is necessarily acyclic. (Can you see why?)

Strong Components and DFS

Strong Components and DFS

- Consider DFS of a digraph.
- Once you enter a strong component, every vertex in the component is reachable.
- So the DFS does not terminate until all the vertices in the component have been visited.
- Thus all vertices in a strong component must appear in the same tree of the DFS forest.

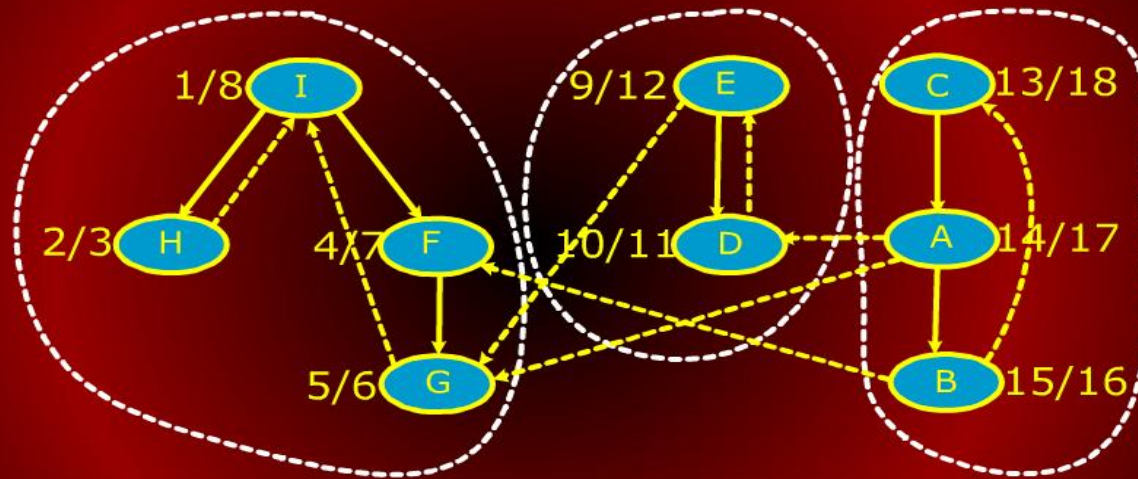
Strong Components and DFS

Strong Components and DFS

- We will discuss the algorithm; the complete proof is in CLR.
- Define G^T to be the digraph with the same vertex set as G but in which all edges have been reversed in direction.
- Given an adjacency list for G , it is possible to compute G^T in $\Theta(V + E)$ time.

Strong Components

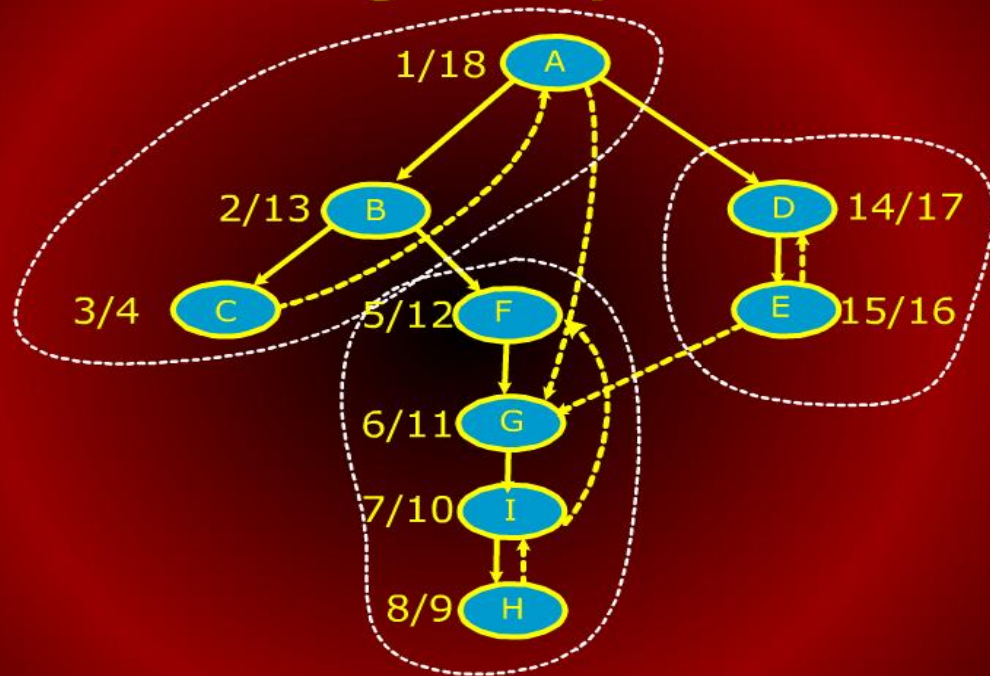
Strong Components



Depth-first search of digraph

Strong Components

Strong Components



Strong Components and DFS

Strong Components and DFS

- Now, run DFS.
- But every time you need a new vertex to start the search from, select the next available vertex according to this reverse topological order of the component DAG.
- This way the search will not “leak out” into other components because other components would already have been visited earlier.

Strong Components

Strong Components



Topological order of component DAG



Reversed topological order

Strong Components and DFS

Strong Components and DFS

- Suppose that you knew the component DAG in advance.
- (This is ridiculous, because you would need to know the strong components and this is the problem we are trying to solve.)
- Further, suppose that you computed a *reversed topological* order on the component DAG.





